

INTERNSHIP PROJECT REPORT

MOVIE RECOMMENDATION SYSTEM

A Personalised Movie Recommendation System

Submitted in partial fulfilment of the requirements for the award of

[YBI Internship]

Submitted by

[Indra karan Nukala]

[2023]

ABSTRACT

The present report documents the design and development of Movie Match, a personalised movie recommendation system developed during the course of an academic internship. The system combines textual metadata (genre, keywords, tagline, cast, and director) into a single feature representation, transforms it using **TF-IDF vectorisation**, and computes pairwise **cosine similarity** between all movies. A user-supplied movie title is first resolved against the dataset using Python's `difflib.get_close_matches` function, which tolerates minor spelling errors or incomplete titles, after which the system returns a ranked list of the most similar films.

The underlying dataset, sourced from the YBI Foundation repository, comprises 4,760 movies described by 21 attributes, including titles, genres, keywords, cast, crew, director, budget, revenue, popularity, and user ratings. This report outlines the background and objectives of the project, describes the dataset and methodology employed, details the implementation process, and discusses the key features and outcomes of the system. The report concludes with an evaluation of the project's achievements and an outline of potential avenues for future development.

Contents

Abstract	1
1 Introduction	3
1.1 Background	3
1.2 Objectives	3
2 Dataset Description	4
3 Methodology	5
3.1 Feature Selection and Combination	5
3.2 TF-IDF Vectorisation	5
3.3 Cosine Similarity	5
3.4 Handling User Input with Close Match	6
3.5 System Workflow	6
4 Implementation	7
4.1 Tools and Technologies Employed	7
4.2 Procedural Steps	7
5 Key Features	9
6 Results and Discussion	10
7 Conclusion	11
8 Future Scope	12

1. Introduction

1.1 Background

In recent years, recommendation systems have become an integral component of digital platforms, enabling users to navigate vast quantities of content with greater ease and efficiency. Within the domain of entertainment, movie recommendation systems play a particularly significant role, assisting users in discovering films that align with their individual preferences amidst an ever-expanding catalogue of available titles.

This project, undertaken as part of an academic internship, seeks to address this challenge through the development of Movie Match, a content-based recommendation system that combines approximate string matching for input handling with TF-IDF and cosine similarity for generating recommendations.

1.2 Objectives

The primary objectives of this internship project are as follows:

- To design and implement a content-based movie recommendation system capable of generating accurate suggestions based on user input.
- To use `difflib.get_close_matches` to resolve minor input variations, such as typographical errors or incomplete titles, to the closest matching entry in the dataset.
- To represent each movie as a combined text feature (genre, keywords, tagline, cast, and director) and vectorise it using TF-IDF.
- To compute cosine similarity across the full movie catalogue and rank recommendations by similarity score.

2. Dataset Description

The dataset employed in this project was obtained from the YBI Foundation repository (*Movies Recommendation.csv*), hosted on GitHub, and comprises **4,760 movies** described across **21 columns**. The principal attributes contained within the dataset are outlined below:

- **Movie_ID, Movie_Title:** Unique identifier and title of each film.
- **Movie_Genre:** One or more genre labels associated with the film (e.g., Crime Comedy, Adventure Action Science Fiction).
- **Movie_Language:** The primary spoken language code of the film.
- **Movie_Budget, Movie_Revenue, Movie_Popularity, Movie_Vote, Movie_Runtime:** Numerical attributes describing the film’s production budget, box-office revenue, popularity score, average user vote, and runtime in minutes.
- **Movie_Release_Date:** The theatrical release date of the film.
- **Movie_Keywords:** A set of descriptive keywords or tags summarising the film’s plot elements.
- **Movie_Overview, Movie_Tagline:** A short synopsis and promotional tagline for the film.
- **Movie_Cast, Movie_Crew, Movie_Director:** The principal cast members, crew details, and director’s name.
- **Movie_Homepage, Movie_Production_House, Movie_Production_Country, Movie_Spoken_Language:** Supplementary metadata used for contextual reference.

For the purpose of building the recommendation engine, five text-based columns – *Movie_Genre*, *Movie_Keywords*, *Movie_Tagline*, *Movie_Cast*, and *Movie_Director* – were selected as the relevant features, since together they best capture the thematic and stylistic identity of each film.

3. Methodology

3.1 Feature Selection and Combination

Rather than relying on numerical attributes such as budget or revenue, Movie Match adopts a **content-based filtering** approach. The five selected text columns are concatenated into a single combined feature string for every movie:

```
x = df['Movie_Genre'] + '_' + df['Movie_Keywords'] + '_' + \
    df['Movie_Tagline'] + '_' + df['Movie_Cast'] + '_' + \
    df['Movie_Director']
```

This produces one descriptive text document per movie that jointly encodes genre, plot keywords, tagline, cast, and director information.

3.2 TF-IDF Vectorisation

The combined text feature is converted into a numerical representation using **Term Frequency–Inverse Document Frequency (TF-IDF)** vectorisation from `scikit-learn`:

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer()
x = tfidf.fit_transform(x)
```

This step transforms the 4,760 combined-text documents into a sparse matrix of shape $4,760 \times 27,466$, where each column corresponds to a unique token weighted by its importance across the corpus.

3.3 Cosine Similarity

Pairwise similarity between every pair of movies is computed using **cosine similarity**, which measures the cosine of the angle between the L2-normalised TF-IDF vectors:

```
from sklearn.metrics.pairwise import cosine_similarity
Similarity_Score = cosine_similarity(x)
```

The result is a $4,760 \times 4,760$ similarity matrix in which entry (i, j) represents how closely related movie i is to movie j based on their combined textual features. A score of 1.0 indicates

identical feature vectors (i.e., the movie compared with itself), while a score of 0.0 indicates no shared terms.

3.4 Handling User Input with Close Match

Since users rarely type movie titles with perfect accuracy, the system uses Python’s built-in `difflib.get_close_matches` function to resolve the user’s input against the list of all movie titles in the dataset:

```
import difflib
Favourite_Movie_Name = input('Enter_your_favourite_movie_name:_')
All_Movies_Title_List = df['Movie_Title'].tolist()
Movie_Recommendation = difflib.get_close_matches(
    Favourite_Movie_Name, All_Movies_Title_List)
Close_Match = Movie_Recommendation[0]
```

For example, entering “*Star Wars*” returns close matches such as *Star Wars*, *Star Trek*, and *State Fair*, with *Star Wars* selected as the top match. The index of this closest-matching movie within the dataset is then retrieved and used to look up its row in the precomputed similarity matrix.

3.5 System Workflow

The end-to-end workflow of Movie Match is as follows:

1. The user provides the title of a favourite movie as input.
2. `difflib.get_close_matches` identifies the closest matching title in the dataset, correcting for minor typos or incomplete titles.
3. The index of the matched movie is used to extract its row from the precomputed $4,760 \times 4,760$ cosine similarity matrix.
4. The similarity scores for all other movies relative to the matched movie are enumerated and sorted in descending order.
5. The system returns the top-ranked movies (by similarity score) as personalised recommendations.

4. Implementation

4.1 Tools and Technologies Employed

The development of Movie Match involved the use of the following tools and technologies:

- **Python**, employed as the primary programming language.
- **Pandas** and **NumPy**, utilised for dataset loading, cleaning, and manipulation.
- **Google Colaboratory**, employed as the development and execution environment.
- **scikit-learn** (`TfidfVectorizer`, `cosine_similarity`), utilised for feature vectorisation and similarity computation.
- **difflib** (Python standard library), utilised for approximate string matching of user input against movie titles.

4.2 Procedural Steps

The implementation of the system followed the procedure outlined below:

1. **Loading the Dataset:** The CSV dataset (4,760 rows, 21 columns) was loaded directly from the YBI Foundation GitHub repository using `pandas.read_csv`.
2. **Feature Selection:** The `Movie_Genre`, `Movie_Keywords`, `Movie_Tagline`, `Movie_Cast`, and `Movie_Director` columns were selected and concatenated into a single combined text feature.
3. **Vectorisation:** The combined feature was transformed into a TF-IDF sparse matrix of shape $4,760 \times 27,466$.
4. **Similarity Computation:** Cosine similarity was computed across the TF-IDF matrix to produce a $4,760 \times 4,760$ similarity score matrix.
5. **Accepting User Input:** The system prompted the user for a favourite movie title and resolved it to the closest dataset entry using `difflib.get_close_matches`.

6. **Generating Recommendations:** The similarity scores for the matched movie were sorted in descending order, and the corresponding movie titles were displayed as the final recommendation list.

A detailed account of the code implementation and accompanying analysis may be found within the associated Colaboratory notebook, *Movie_Recommendation_System_Colab.ipynb*, accessible via the project repository.

5. Key Features

The Movie Match system incorporates the following notable features:

- **Typo-Tolerant Input:** The use of `difflib.get_close_matches` allows the system to gracefully handle typographical errors and partial titles.
- **Content-Based Recommendation:** TF-IDF weighting combined with cosine similarity ensures that recommendations reflect genuine thematic overlap in genre, keywords, tagline, cast, and director rather than popularity alone.
- **Full-Catalogue Ranking:** Every one of the 4,760 movies in the dataset is scored and ranked relative to the user's chosen film, enabling a comprehensive and personalised recommendation list.
- **Lightweight and Reproducible:** The entire pipeline runs within a single Google Colab notebook using only pandas, NumPy, scikit-learn, and difflib, with no external model training required.

6. Results and Discussion

When tested with the query “*Star Wars*”, the Close Match step correctly resolved the input to the exact title *Star Wars* (dataset index 2) despite the availability of visually similar alternatives such as *Star Trek* and *State Fair*. The subsequent similarity ranking, generated by sorting the cosine similarity scores for index 2 in descending order, placed *Star Wars* itself at a similarity score of 1.0, followed by other science-fiction and adventure titles with progressively lower scores (e.g., 0.132, 0.124, 0.119), reflecting shared genre, keyword, and cast overlap.

These results demonstrate that the combination of TF-IDF-based content representation and cosine similarity produces recommendations that are thematically coherent, while the Close Match step ensures the system remains robust to imperfect or informally typed user input. Overall, the results obtained during testing indicate that the system successfully fulfils its intended objective of providing personalised and reliable movie recommendations to its users.

7. Conclusion

This internship project has resulted in the successful design and implementation of Movie Match, a personalised, content-based movie recommendation system. By combining genre, keyword, tagline, cast, and director information into a TF-IDF-weighted feature space and ranking movies via cosine similarity, the system is capable of delivering accurate and relevant recommendations. The integration of `difflib.get_close_matches` further ensures that the system remains usable even when the user's input contains minor spelling errors.

The project has provided valuable practical experience in the domains of text vectorisation, similarity-based recommendation, and data analysis, thereby contributing meaningfully to the author's academic and professional development.

8. Future Scope

While the current implementation of Movie Match fulfils its core objectives, several opportunities for future enhancement have been identified, including:

- The incorporation of additional filtering criteria, such as release year, runtime, or user vote, in order to further refine recommendations.
- The integration of collaborative filtering techniques (based on user ratings) to complement the existing content-based approach.
- Replacing TF-IDF with dense semantic embeddings (e.g., word or sentence embeddings) to capture deeper contextual similarity between movies.
- The development of a graphical or web-based user interface to enhance overall usability and accessibility.
- The expansion of the underlying dataset to encompass a broader range of films and user ratings.

Bibliography

- [1] YBI Foundation. *Movies Recommendation Dataset*. Available at: <https://github.com/YBI-Foundation/Dataset/raw/main/Movies%20Recommendation.csv>
- [2] Bhutada, V. *Movie Recommendation System – Colaboratory Notebook*. Available at: <https://github.com/virajbhutada/Movie-Recommendation-System>